Platform Intelligence Engine (PIE)

Technical Documentation

Package: adpie (v0.3.0) | Generated August 27, 2026

PIE - Technical Documentation

- Platform Intelligence Engine (PIE) — Technical Documentation
  - 1. Executive Summary
  - 2. System Architecture
    - 2.1 High-Level Architecture
    - 2.2 Data / Reasoning Pipeline
    - 2.3 Hybrid Query Decision Model
    - 2.4 Architectural Principles
  - 3. Technology Stack
  - 4. Repository Layout
  - 5. Backend Module Reference (`src/pie/`)
    - 5.1 `pie.core` — Foundation
    - 5.2 `pie.auth` — Identity & RBAC
    - 5.3 `pie.discovery` — Metadata Extraction & Storage
    - 5.4 `pie.graph` — Knowledge Graph & Intelligence Engines
    - 5.5 `pie.context` — Token Budgeting & Prompt Packaging
    - 5.6 `pie.ai` — Reasoning Engine
    - 5.7 `pie.teams` — Microsoft Teams Integration
    - 5.8 `pie.api` — FastAPI Application
    - 5.9 `pie.cli` — Command-Line Interface
  - 6. REST API Reference (base path `/api/v1`)
    - 6.1 Authentication — `/api/v1/auth` (tag: *Authentication & Identity*)
    - 6.2 Discovery & Asset Catalog — `/api/v1` (tag: *Discovery & Asset Catalog*, no extra prefix)
    - 6.3 Knowledge Graph — `/api/v1/graph` (tag: *Knowledge Graph & Lineage Traversal*)
    - 6.4 Audit & Governance — `/api/v1/audit` (tag: *Technical Debt & Governance Auditing*)
    - 6.5 AI Reasoning — `/api/v1/ai` (tag: *AI Reasoning & Engineering Assistant*)
    - 6.6 Settings — `/api/v1/settings`
    - 6.7 Microsoft Teams — `/api/v1/teams` (tag: *Microsoft Teams Bot & Webhooks*)
    - 6.8 Health & Diagnostics
  - 7. Frontend Architecture (`frontend/`)
  - 8. Data & Domain Model Summary
    - 8.1 Knowledge Graph Node Types
    - 8.2 Knowledge Graph Edge Types
    - 8.3 Confidence Levels for Expression-Derived Edges
    - 8.4 Change Impact Analysis — 13-Step Pipeline
  - 9. Testing
  - 10. Configuration Reference
  - 11. Security Notes
  - 12. Known Documentation Assets Already in the Repo
  - 13. Versioning & Status Snapshot

Platform Intelligence Engine (PIE) — Technical Documentation

Package name: `adpie` (import name `pie`) · **Version:** 0.3.0 · **Repo root folder:** `PlatformIntelligenceEngine-main`

---

1. Executive Summary

Platform Intelligence Engine (PIE) is an AI-assisted **engineering intelligence layer for Azure Data Factory (ADF)**. It connects to Azure with **read-only (Reader) RBAC**, discovers ADF metadata (pipelines, activities, datasets, linked services, triggers, data flows), normalizes it into typed domain models, builds an **in-memory knowledge graph**, and layers **deterministic graph traversal** and **grounded LLM reasoning** on top of it to answer engineering questions such as:

- “Explain pipeline `Customer_Load` .”

- “What breaks if I delete `CustomerDataset`?”
- “Which pipelines read from SAP or write to Azure SQL?”
- “Show me the technical debt and orphaned pipelines.”

PIE explicitly is **not** a pipeline editor/execution platform — it never mutates Azure resources.

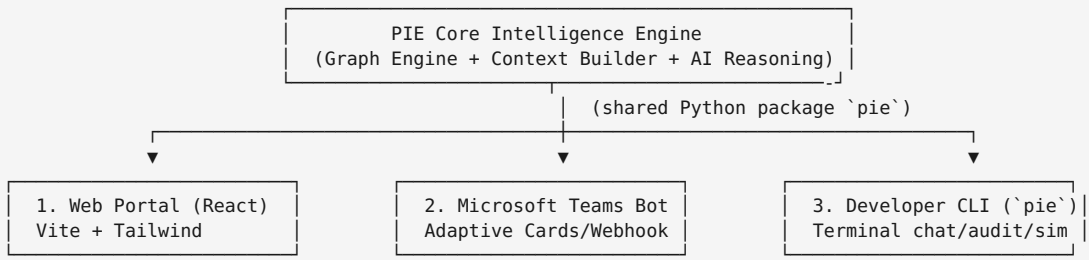
It ships as: 1. A **FastAPI backend** (`src/pie/`) exposing a versioned REST API (`/api/v1/...`) with SSE streaming chat. 2. A **React + Vite frontend** (`frontend/`) — the “Asset Explorer” web portal. 3. A **Typer/argparse CLI** (`pie` console script) for headless discovery, audits, deletion simulation, and chat. 4. A **Microsoft Teams bot integration** (Adaptive Cards + webhook) built on the same core engine.

All three channels (Web, CLI, Teams) sit on top of one shared, headless core engine — there is a single source of truth for graph, context, and AI reasoning logic.

---

2. System Architecture

2.1 High-Level Architecture



2.2 Data / Reasoning Pipeline

```
Azure Data Factory (ARM/Mgmt API, Reader RBAC)
        |  azure-identity / azure-mgmt-datafactory / MSAL PKCE
        ▼
Discovery Engine (pie.discovery.extractor)
        |  paginated REST calls, secret sanitization
        ▼
Metadata Normalizer (pie.discovery.normalizer)  ──►  In-Memory Repository (pie.discovery.repository)
        |  typed Pydantic domain models                    (thread-safe cache, disk persistence)
        ▼
Knowledge Graph Builder (pie.graph.builder)
        |  typed directed multigraph (Nodes: Pipeline/Activity/Dataset/LinkedService/Trigger;
        |   Edges: USES/CONTAINS/DEPENDS_ON/READS/WRITES/CALLS/EXECUTES/TRIGGERED_BY)
        ▼
Graph Services: Traversal · Storyteller · QueryEngine · DeletionSimulator ·
                ChangeImpactEngine · AuditEngine · ExpressionAnalyzer
        ▼
Context Builder (pie.context) — subgraph extraction, schema compression (90%+ token
        |   reduction), token budgeting, multi-intent prompt packaging
        ▼
AI Reasoning Engine (pie.ai.engine) — intent routing, LLM provider abstraction,
        |   grounded prompt assembly, streaming responses
        ▼
REST API (pie.api) ──► React Web UI / Teams Bot / CLI
```

2.3 Hybrid Query Decision Model

Query type	Example	Path
Deterministic (target asset known)	“Explain Customer_Load”, “What happens if I delete Dataset Y?”	Graph → Traversal → Context Builder → LLM → Response
Discovery / exploratory (asset unknown)	“Which pipelines use SAP?”, “Show disabled triggers”	Search/Query Engine → Filtered assets → Graph → Context Builder → LLM → Response

2.4 Architectural Principles

1. **Metadata-first** — discovered metadata is the single source of truth (no manual entry).
2. **Knowledge-graph centric** — the graph is the canonical structural relationship layer.
3. **Read-only / least privilege** — Azure **Reader** role only; PIE never writes to Azure.
4. **AI for reasoning, not storage** — the LLM interprets structured context; it doesn't memorize topology.
5. **Deterministic vs. semantic separation** — known-asset questions use graph traversal; exploratory questions use search/query engines.
6. **Zero hallucination tolerance** — the Context Builder strictly bounds what is sent to the LLM to extracted metadata; prompts instruct the model to answer only from grounded context.

---

3. Technology Stack

Layer	Technology
Backend language/runtime	Python ≥ 3.10 (developed/tested on 3.11)
Web framework	FastAPI ≥ 0.110, Uvicorn ≥ 0.28 (ASGI)
Data validation	Pydantic ≥ 2.5, pydantic-settings ≥ 2.1
Azure SDKs	`azure-identity`, `azure-mgmt-resource`, `azure-mgmt-datafactory`, `azure-mgmt-subscription`
Auth	MSAL (`msal` ≥ 1.28) — OAuth2 PKCE browser login against the Azure CLI public client
HTTP client	`httpx`, `requests`
LLM integration	`openai` SDK (used for OpenAI, Azure OpenAI/AI Foundry, and OpenAI-compatible providers e.g. NVIDIA NIM); provider abstraction also lists Anthropic/Gemini
CLI/console UX	`rich`, `argparse`
Testing	`pytest`, `pytest-asyncio`, `httpx` test client
Packaging	`setuptools` (src-layout), `pyproject.toml`, console script `pie`
Frontend framework	React 19 + Vite 8
Frontend styling	Tailwind CSS 3.4 + PostCSS/Autoprefixer
Frontend routing/http	`react-router-dom` 7, `axios`
Frontend markdown/icons	`react-markdown` + `remark-gfm`, `lucide-react`
Frontend linting	`oxlint`

---

4. Repository Layout

```
PlatformIntelligenceEngine-main/
├── setup.ps1 / setup.sh          # One-shot bootstrap (venv + pip install -e . + npm install)
├── start.ps1 / start.sh          # Launch backend + frontend together
├── pyproject.toml                # Package metadata, dependencies, console script `pie`
├── requirements.txt              # Pinned backend dependency floor
├── .env.example                  # Environment variable template
├── check_repo.py                 # Repo/health sanity-check script
├── test_factory_workflow.py      # Ad hoc top-level integration/smoke script
├── test_pipeline_queries.py      # Ad hoc top-level integration/smoke script
├── test_workflow_detailed.py     # Ad hoc top-level integration/smoke script
├── src/
│   ├── adpie.egg-info/           # Editable-install build metadata
│   └── pie/                      # ── Backend application package ──
│       ├── core/                 # Settings, logging, exceptions
│       ├── auth/                 # Azure Identity, MSAL PKCE, RBAC discovery, sessions
│       ├── discovery/            # ADF metadata extraction, normalization, repository/cache
│       ├── graph/                # Knowledge graph, traversal, storyteller, audits, impact
│       ├── context/              # Subgraph compression + token budgeting + prompt packaging
│       ├── ai/                   # LLM providers, intent router, reasoning engine, chat sessions
│       ├── teams/                # Microsoft Teams Adaptive Card builders
│       ├── api/                  # FastAPI app, routers, DTOs
│       └── cli.py                # `pie` console-script entry point
├── frontend/                     # ── React + Vite web portal ──
│   └── src/
│       ├── api/client.js         # Axios client (baseURL, session header)
│       ├── components/
│       │   ├── Chat/             # AI chat UI (streaming, markdown rendering)
│       │   ├── Explorer/         # Asset explorer / data canvas / factory overview
│       │   ├── ImpactAnalysis/   # Change-impact / deletion-risk panel
│       │   ├── Onboarding/       # Login → subscription → factory setup wizard
│       │   └── Sidebar/          # Navigation + settings modal (API keys)
│       ├── layouts/MainWorkspace.jsx
│       └── App.jsx / main.jsx
├── spikes/                       # Phase-2 proof-of-concept scripts (1 per subsystem) + captured output JSON
│   ├── spike_1_auth/             # Azure auth + RBAC discovery PoC
│   ├── spike_2_discovery/        # ADF metadata extraction PoC
│   ├── spike_3_graph/            # Knowledge graph PoC
│   ├── spike_4_context/          # Context builder / token budgeting PoC
│   └── spike_5_ai/               # AI reasoning engine PoC
├── tests/
│   ├── api/                      # FastAPI TestClient-based endpoint tests (8 files)
│   └── unit/                     # Unit tests for graph, context, auth, discovery, AI (8 files)
└── Documentation/
    ├── PIE_MASTER_CONTEXT.md     # Master architecture & phase blueprint (source of truth)
    ├── Phases/                   # Phase 1–7 planning docs (Product → Enterprise Roadmap)
    └── Requirement/               # Original .docx requirement documents per phase
```

Two duplicate copies of the master context file exist: `./PIE_MASTER_CONTEXT.md` (repo root) and `./Documentation/PIE_MASTER_CONTEXT.md` — keep these in sync manually if edited.

---

5. Backend Module Reference (`src/pie/`)

5.1 `pie.core` — Foundation

File	Responsibility
`config.py`	`Settings` (Pydantic `BaseSettings`) loaded from `.env` /environment. Defines `AuthMode` enum (`interactive`, `device_code`, `default`, `cli`, `service_principal`, `mock`), Azure Entra credentials, optional subscription/resource-group/factory scope, Azure AI Foundry / AI Search settings, log level, output dir. Exposes cached singleton `get_settings()`.
`exceptions.py`	Domain exception hierarchy: `PieError` (base, carries `message` + `details`), `PieAuthError`, `PiePermissionError`, `PieDiscoveryError`, `PieResourceNotFoundError`.
`logging.py`	Rich-themed structured logging (`get_logger`) and a shared `console` (Rich `Console`) for CLI output.

5.2 `pie.auth` — Identity & RBAC

File	Responsibility
`credentials.py`	`CredentialFactory` — builds an Azure `TokenCredential` per `AuthMode` (`DefaultAzureCredential` , `InteractiveBrowserCredential` , `DeviceCodeCredential` , `AzureCliCredential` , `ClientSecretCredential`); `MockTokenCredential` for offline/demo mode.
`token_manager.py`	`EntraTokenManager` — MSAL-based token acquisition/persistence and direct REST RBAC querying; `BearerTokenCredential` wraps a raw bearer token as an Azure SDK `TokenCredential` .
`rbac_discovery.py`	`AzureRbacDiscovery` — enumerates accessible subscriptions, resource groups, and Data Factories under the caller's Reader role via `SubscriptionClient` , `ResourceManagementClient` , `DataFactoryManagementClient` .
`session_store.py`	In-memory `SessionStore` for OAuth2 PKCE flows: tracks pending PKCE state → code_verifier mappings and active sessions (`session_token` → claims + access token). Documented as **Phase 3 in-process store**; migration to Redis/Azure Cache is called out as a Phase 5 item for HA deployments.
`models.py`	`TenantContext` , `SubscriptionMetadata` , `ResourceGroupMetadata` , `DataFactoryBrief` , `AuthContext` , `Spike1Result` .

Auth flow (browser OAuth2 PKCE against the Azure CLI public client): 1. `POST /api/v1/auth/login` — generates PKCE parameters, stores `state → flow_id` , returns a Microsoft login URL. 2. User completes login in the browser; Microsoft redirects to a **persistent local callback HTTP server on port 8100** (`http://localhost:8100` , registered against Azure CLI's public client ID `04b07795-8ddb-461a-bbee-02f9e1bf7b46`). 3. The callback server exchanges the authorization code for tokens and creates a PIE session. 4. Frontend polls `GET /api/v1/auth/poll/{flow_id}` until the session token is ready. 5. `GET /api/v1/auth/session` (with `X-Session-Token` header) inspects/validates the session. 6. `POST /api/v1/auth/logout` revokes the session.

Scopes requested: `openid profile email https://management.azure.com/user_impersonation` .

5.3 `pie.discovery` — Metadata Extraction & Storage

File	Responsibility
`extractor.py`	`AzureDataFactoryExtractor` — pulls pipelines, datasets, linked services, triggers, and data flows from ADF via paginated ARM REST calls; wraps failures in `PieDiscoveryError` .
`normalizer.py`	Translates heterogeneous raw ADF JSON into unified Pydantic domain models (activities, retry policies, parameters/variables at 3 tiers).
`models.py`	Domain models: `ParameterDefinition` , `VariableDefinition` , `RetryPolicy` , `ActivityMetadata` , `PipelineMetadata` , `DatasetMetadata` , `LinkedServiceMetadata` , `TriggerMetadata` , `DataFlowMetadata` , `FactoryMetadata` , `Spike2Result` .
`repository.py`	Thread-safe (`threading.Lock`) **in-memory metadata repository** with disk-cache persistence/restore (`load_cached_factories`), keyed by factory + subscription; the single source of truth queried by the API layer and CLI.

3-tier parameter hierarchy normalized by this layer: - Tier 1 — Factory global parameters (`@pipeline().globalParameters.xxx`) - Tier 2 — Pipeline parameters (`@pipeline().parameters.xxx`) - Tier 3 — Pipeline variables (`@variables('xxx')`)

5.4 `pie.graph` — Knowledge Graph & Intelligence Engines

File	Responsibility
`models.py`	`NodeType` (`Pipeline`, `Activity`, `Dataset`, `LinkedService`, `Trigger`), `EdgeType`, `GraphNode`, `GraphEdge`, `Subgraph`, `ImpactReport`, `ChangeType`, `ChangeRequest`, `ConfidenceLevel`, `ExpressionReference`, `Spike3Result`.
`builder.py`	`KnowledgeGraph` / `KnowledgeGraphBuilder` — constructs the typed directed multigraph from normalized `FactoryMetadata`, with O(1) node lookups.
`expression_analyzer.py`	`ExpressionAnalyzer` — regex-based detector for ADF dynamic-content expression references (`@activity('X').output...`, `dataset('X')`, `@pipeline().parameters.X`) that represent high-confidence data dependencies not always captured by structural edges.
`traversal.py`	`GraphTraversalService` — deterministic BFS/DFS traversal: upstream lineage, downstream blast radius, k-hop subgraph extraction, cycle detection.
`storyteller.py`	`PipelineStoryteller` — converts raw activity topology into a plain-language, ordered walkthrough (SQL text, stored-proc names, Key Vault references, REST endpoints, copy source/sink).
`query_engine.py`	`AssetQueryEngine` — multi-criteria discovery (file format, on-prem/cloud connectivity, folder, schema/columns) with fuzzy matching (`difflib`).
`deletion_simulator.py`	`AssetDeletionSimulator` — deterministic what-if deletion analysis: broken readers/writers, cascade failures, step-by-step remediation plan.
`change_impact_engine.py`	`ChangeImpactEngine` — the flagship **13-step deterministic impact-analysis pipeline**: (1) identify target → (2) identify change → (3) resolve metadata → (4) direct deps → (5) expression refs → (6) downstream graph → (7) upstream graph → (8) external systems → (9) affected workflows → (10) build evidence → (11) risk scoring → (12) recommendation → (13) summary.
`audit_engine.py`	`SecurityAndGovernanceAuditor` + technical-debt/orphan detection + SaaS-vendor mapping (SAP, Dynamics CRM, Databricks, Coupa, OpenText, Datex, etc.) + Key Vault compliance checks + schedule-concurrency heatmap generation.

5.5 `pie.context` — Token Budgeting & Prompt Packaging

File	Responsibility
`models.py`	`TokenBudget` (default `max_tokens=4000`, section allocation: 40% activity flow / 30% schema / 20% lineage / 10% linked services), `ContextPackage`, `Spike4Result`.
`compressor.py`	`SchemaCompressor` — strips UI layout coordinates, system GUIDs, and empty metadata; compresses raw activity/asset JSON into dense single-line semantic representations (validated 90–99%+ token reduction).
`budgeter.py`	`TokenBudgeter` — `estimate_tokens()` heuristic (~4 chars/token) and strict per-section budget enforcement to avoid truncation/hallucination.
`builder.py`	`ContextBuilder` — orchestrates subgraph extraction + compression + budgeting + Markdown formatting into an LLM-ready context payload.
`intent_builder.py`	`MultiIntentContextBuilder` + `ContextIntent` enum (`ARCHITECTURE`, `DEBUGGING`, `IMPACT_ANALYSIS`, `MODERNIZATION`) — tailors which sections/detail level are included per intent.

5.6 `pie.ai` — Reasoning Engine

File	Responsibility
`models.py`	`ChatRole` (`system` / `user` / `assistant`), `ChatMessage`, `QueryIntent`, `LLMProviderType`, `LLMConfig`, `ReasoningResponse`, `Spike5Result`.
`providers.py`	`BaseLLMProvider` (ABC) + concrete providers for **OpenAI, Azure OpenAI/AI Foundry, Anthropic, Gemini, NVIDIA NIM (OpenAI-compatible)**, plus `DeterministicMockLLMProvider` — a sub-300ms, zero-cost offline provider used for tests/demos when no API key is configured. `create_llm_provider()` factory selects the provider from `LLMConfig`.
`router.py`	`QueryIntentRouter` — classifies free-text questions into 6 cognitive intents (`ARCHITECTURE`, `DEBUGGING`, `IMPACT`, `SEARCH`, `CODE_GEN`, `SECURITY_AUDIT`) and extracts the likely target asset name via stop-word filtering + fuzzy matching.
`prompts.py`	Version-controlled prompt template library: Documentation, Business Summary, Technical Summary, Architecture Review, Impact Analysis, Best Practices/Recommendation prompts. Includes an activity-type → plain-English translation guide (Copy Activity, Web/REST, Lookup, Stored Procedure, ForEach, If Condition, etc.).
`engine.py`	`PIEReasoningEngine` — the master orchestrator: routes intent, builds grounded context, selects/calls the LLM provider, supports both single-shot (`/ai/ask`) and streaming (`/ai/chat/stream`) responses; rebuilds the in-memory graph when the requested factory differs from the currently loaded one.
`chat_session.py`	`ChatSession` / in-memory chat session store — bounded, TTL-expiring multi-turn conversation history keyed by session token + frontend chat-session id.

5.7 `pie.teams` — Microsoft Teams Integration

File	Responsibility
`adaptive_cards.py`	`TeamsAdaptiveCardBuilder` — generates schema-compliant Adaptive Cards v1.4 (subscription-selection cards, deletion-impact cards, Q&A response cards) for the Teams bot webhook channel.

5.8 `pie.api` — FastAPI Application

File	Responsibility
`app.py`	Application factory (`create_app`). Configures CORS (`allow_origins=["*"]`), global exception handlers (`PieError` → HTTP 400, unhandled → HTTP 500), `/health` endpoint, `lifespan` hook (preloads cached factories into the repository on startup, starts/stops the OAuth callback server on port 8100), registers all routers under `/api/v1` . OpenAPI at `/api/v1/openapi.json` , Swagger UI at `/docs` , ReDoc at `/redoc` .
`dependencies.py`	Shared FastAPI dependency injectors (session/auth extraction, repository/graph accessors).
`models.py`	All request/response DTOs (Pydantic v2) — session, discovery, graph, audit, AI, Teams schemas.
`routers/auth.py`	OAuth2 PKCE login/poll/session/logout endpoints + persistent callback HTTP server lifecycle.
`routers/discovery.py`	Subscription/factory discovery, sync, and asset catalog listing/detail endpoints.
`routers/graph.py`	Topology, lineage, subgraph, deletion-simulation, and change-impact endpoints.
`routers/audit.py`	Technical-debt, concurrency-heatmap, SaaS-vendor-map, and parameter-audit endpoints.
`routers/ai.py`	`/ai/ask` (single-shot), `/ai/chat/stream` (SSE streaming), `/ai/generate-code` (modernization code-gen), `/ai/insights` (factory-level AI insights).
`routers/settings.py`	Get/save API keys for LLM providers (`/settings/keys`).
`routers/teams.py`	Teams bot webhook handler + deletion-impact Adaptive Card generator.

5.9 `pie.cli` — Command-Line Interface

Console script `pie` (registered in `pyproject.toml` as `pie = "pie.cli:main"`). Subcommands:

Command	Purpose
`pie serve [--host] [--port] [--reload]`	Starts the FastAPI server via `uvicorn.run("pie.api.app:app", ...)` .
`pie discover`	Interactive hierarchical subscription/factory discovery; prints a Rich table of currently synced factories.
`pie audit`	Runs technical-debt, SaaS-vendor, and schedule-concurrency audits against all loaded factories; prints a summary panel.
`pie simulate-deletion <asset>`	Runs the what-if deletion simulator for a named asset and renders a risk table.
`pie chat`	Launches a conversational AI terminal assistant grounded in loaded factory metadata.
`pie clear`	Clears all loaded factories from the in-memory workspace.

---

6. REST API Reference (base path `/api/v1`)

All routes are registered without a global auth dependency at the router level; endpoints that need a session read it via `X-Session-Token` header parameters (see `routers/auth.py` , `dependencies.py`).

6.1 Authentication — `/api/v1/auth` (tag: *Authentication & Identity*)

Method	Path	Description
POST	`/auth/login`	Start OAuth2 PKCE flow; returns Microsoft login URL. Query param `tenant` (default `common`).
GET	`/auth/poll/{flow_id}`	Poll a pending login flow until `status == "complete"` and a `session_token` is issued.
GET	`/auth/session`	Return current session info (`SessionInfo`) — requires `X-Session-Token`.
POST	`/auth/logout`	Revoke the current session.

6.2 Discovery & Asset Catalog — `/api/v1` (tag: *Discovery & Asset Catalog*, no extra prefix)

Method	Path	Description
GET	`/subscriptions`	List Azure subscriptions accessible to the authenticated user.
POST	`/subscriptions/factories`	List Data Factories within one or more selected subscriptions.
POST	`/discovery/sync`	Discover + extract + normalize + cache metadata for selected factories (persists to disk cache).
GET	`/factories`	List currently loaded (in-memory/cached) factories.
GET	`/factories/{name}/summary`	Summary stats for a factory (counts of pipelines/datasets/etc.).
POST	`/factories/{name}/refresh`	Re-sync a single factory's metadata from Azure.
GET	`/pipelines`	List all pipelines across loaded factories.
GET	`/pipelines/{name}`	Detailed pipeline metadata (activities, parameters, variables).
GET	`/datasets`	List all datasets.
GET	`/linked-services`	List all linked services.
GET	`/triggers`	List all triggers.
GET	`/factories/{name}/pipelines`	Pipelines scoped to a specific factory.
GET	`/factories/{name}/global-parameters`	Factory-level (Tier 1) global parameters.

6.3 Knowledge Graph — `/api/v1/graph` (tag: *Knowledge Graph & Lineage Traversal*)

Method	Path	Description
GET	`/graph/topology`	Full graph topology summary (node/edge counts by type).
GET	`/graph/lineage/{asset_name}`	Upstream/downstream lineage for a named asset.
GET	`/graph/subgraph/{asset_name}`	K-hop subgraph centered on an asset (for visualization).
POST	`/graph/deletion-simulation`	What-if deletion simulation — cascading breakage + remediation plan.
POST	`/graph/change-impact`	13-step deterministic change-impact analysis for a proposed modification.

6.4 Audit & Governance — `/api/v1/audit` (tag: *Technical Debt & Governance Auditing*)

Method	Path	Description
GET	`/audit/technical-debt`	Orphan pipelines + zero-retry fragile activities report.
GET	`/audit/concurrency-heatmap`	Schedule-collision heatmap (concurrent pipeline execution windows).
GET	`/audit/saas-vendors`	External SaaS vendor/system integration map (SAP, Dynamics, Databricks, etc.).
GET	`/audit/parameters`	Parameter audit across the 3-tier hierarchy (globals/params/variables).

6.5 AI Reasoning — `/api/v1/ai` (tag: *AI Reasoning & Engineering Assistant*)

Method	Path	Description
POST	`/ai/ask`	Single-shot grounded Q&A. Body includes `query`, `factory_name`, `model`. Returns `AIAskResponse` (detected intent, markdown response, latency).
POST	`/ai/chat/stream`	Server-Sent Events streaming chat (token-by-token) for the same reasoning pipeline.
POST	`/ai/generate-code`	AI-assisted modernization code generation (e.g., PySpark/dbt translation of an ADF pipeline).
POST	`/ai/insights`	Factory-level AI-generated insights summary.

6.6 Settings — `/api/v1/settings`

Method	Path	Description
GET	`/settings/keys`	Retrieve stored LLM provider API keys (masked).
POST	`/settings/keys`	Save/update LLM provider API keys used by `pie.ai.providers`.

6.7 Microsoft Teams — `/api/v1/teams` (tag: *Microsoft Teams Bot & Webhooks*)

Method	Path	Description
POST	`/teams/webhook`	Inbound Teams bot activity webhook handler.
POST	`/teams/cards/deletion-impact`	Generate a Teams Adaptive Card summarizing a deletion-impact result.

6.8 Health & Diagnostics

Method	Path	Description
GET	`/health`	Liveness check — returns `{"status": "HEALTHY", "service": "PIE-Core-Platform", "version": "3.0.0"}`.

Full interactive documentation is always available at runtime: - Swagger UI: `http://localhost:8000/docs` - ReDoc: `http://localhost:8000/redoc` - Raw OpenAPI schema: `http://localhost:8000/api/v1/openapi.json`

---

7. Frontend Architecture (`frontend/`)

React 19 SPA built with Vite 8 and styled with Tailwind CSS.

Path	Responsibility
`src/main.jsx`	React root bootstrap.
`src/App.jsx`	Top-level app shell/router mount.
`src/api/client.js`	Axios instance — base URL pointed at the FastAPI backend, attaches `X-Session-Token` from `localStorage`.
`src/layouts/MainWorkspace.jsx`	Primary authenticated workspace layout (sidebar + main content + chat drawer).
`src/components/Onboarding/SetupWizard.jsx`	Login → subscription selection → factory selection → sync flow. Always re-runs subscription/factory selection after every login (see §10.3).
`src/components/Sidebar/Sidebar.jsx`, `SettingsModal.jsx`	Navigation and LLM-provider API key settings UI.
`src/components/TopBar.jsx`	Top navigation bar (factory switcher, model selector).
`src/components/Explorer/FactoryOverview.jsx`, `DataCanvas.jsx`	Asset explorer views — factory summary and interactive dependency-graph canvas.
`src/components/ImpactAnalysis/ChangeImpactPanel.jsx`	UI for the deletion/change-impact simulation flow.
`src/components/Chat/ChatContainer.jsx`, `MarkdownMessage.jsx`	AI copilot chat UI; streams `/ai/chat/stream` and renders Markdown responses (tables, code blocks) via `react-markdown` + `remark-gfm`.

Default dev ports: **backend** `:8000`, **frontend** `:5173` (Vite default). CORS on the backend is fully open (`*`) for local development.

---

8. Data & Domain Model Summary

8.1 Knowledge Graph Node Types

`Pipeline`, `Activity`, `Dataset`, `LinkedService`, `Trigger` (plus Data Flows referenced through activities).

8.2 Knowledge Graph Edge Types

`USES`, `CONTAINS`, `DEPENDS_ON`, `READS`, `WRITES`, `CALLS`, `EXECUTES`, `TRIGGERED_BY` — 8 relationship types.

8.3 Confidence Levels for Expression-Derived Edges

`ExpressionAnalyzer` assigns a `ConfidenceLevel` (structural edges from the ADF JSON schema itself are highest-confidence; expression-parsed references such as `@activity('X').output` are marked accordingly) so downstream consumers can distinguish guaranteed vs. inferred dependencies.

8.4 Change Impact Analysis — 13-Step Pipeline

1. Identify target asset
2. Identify proposed change type (`ChangeType`)
3. Resolve target metadata
4. Compute direct dependencies
5. Scan expression references
6. Traverse downstream graph
7. Traverse upstream graph
8. Identify external systems touched
9. Identify affected business workflows
10. Assemble supporting evidence
11. Compute risk score
12. Generate remediation/recommendation
13. Produce human-readable summary

---

9. Testing

Directory	Scope
`tests/api/`	FastAPI `TestClient`-driven endpoint tests: `test_api_ai.py`, `test_api_audit.py`, `test_api_auth.py`, `test_api_discovery.py`, `test_api_graph.py`, `test_api_security_isolation.py`, `test_api_teams.py`, `test_phase3_complete.py` (`conftest.py` supplies shared fixtures).
`tests/unit/`	Unit-level coverage: `test_ai_reasoning.py`, `test_auth_models.py`, `test_change_impact.py`, `test_context_builder.py`, `test_credentials.py`, `test_discovery_mock_fixture.py`, `test_discovery_normalizer.py`, `test_graph_engine.py`.
Top-level ad hoc scripts	`test_factory_workflow.py`, `test_pipeline_queries.py`, `test_workflow_detailed.py` — manual/exploratory integration scripts (not part of the `pytest` discovery path by convention, but runnable standalone).

Total discovered `pytest` test functions: 152 across `tests/api` and `tests/unit`.

Run the full suite:

```
.venv/bin/python -m pytest tests/ -v
```

The `DeterministicMockLLMProvider` (in `pie.ai.providers`) allows the entire AI reasoning pipeline to be exercised in CI without real Azure/LLM credentials — it returns deterministic, sub-300ms responses.

---

10. Configuration Reference

Configuration is loaded by `pie.core.config.Settings` from environment variables or a `.env` file at the repo root (`model_config` also references a `src/.env` used during local development — keep secrets out of version control).

Variable	Aliases	Default	Purpose
`PIE_AUTH_MODE`	`AZURE_AUTH_MODE`	`default`	One of `interactive`, `device_code`, `default`, `cli`, `service_principal`, `mock`.
`AZURE_TENANT_ID`	`TENANT_ID`	—	Microsoft Entra tenant ID.
`AZURE_CLIENT_ID`	`CLIENT_ID`	—	Service principal App ID (only needed for `service_principal` mode).
`AZURE_CLIENT_SECRET`	`CLIENT_SECRET`	—	Service principal secret.
`AZURE_SUBSCRIPTION_ID`	`SUBSCRIPTION_ID`	—	Optional default subscription scope (auto-discovers if empty).
`AZURE_RESOURCE_GROUP`	`RESOURCE_GROUP`	—	Optional default resource group scope.
`AZURE_FACTORY_NAME`	`FACTORY_NAME`	—	Optional default Data Factory scope.
`AZURE_AI_FOUNDRY_ENDPOINT`	`AI_FOUNDRY_ENDPOINT`	—	Azure AI Foundry / Azure OpenAI endpoint.
`AZURE_AI_FOUNDRY_KEY`	`AI_FOUNDRY_KEY`	—	Azure AI Foundry / Azure OpenAI key.
`AZURE_AI_FOUNDRY_MODEL`	`AI_FOUNDRY_MODEL`	`gpt-4o-mini`	Deployed model name.
`AZURE_AI_SEARCH_ENDPOINT`	`AI_SEARCH_ENDPOINT`	—	Azure AI Search endpoint (semantic discovery).
`AZURE_AI_SEARCH_KEY`	`AI_SEARCH_KEY`	—	Azure AI Search key.
`AZURE_AI_SEARCH_INDEX_NAME`	`AI_SEARCH_INDEX_NAME`	`pie-adf-metadata-index`	Search index name.
`PIE_LOG_LEVEL`	`LOG_LEVEL`	`INFO`	Log verbosity.
`PIE_OUTPUT_DIR`	`OUTPUT_DIR`	`output`	Directory for exported docs/cache artifacts.

LLM provider API keys used by the chat/AI features (OpenAI, Anthropic, Gemini, NVIDIA NIM) can alternatively be supplied at runtime through the **Settings modal in the web UI**, persisted via `POST /api/v1/settings/keys`, rather than environment variables.

---

11. Security Notes

- **Least privilege:** designed to operate with only the Azure built-in **Reader** role on target subscriptions — no write/deploy permissions are requested or required.
- **No pipeline execution:** PIE is explicitly out-of-band; it cannot trigger, edit, or deploy ADF pipelines.
- **Secret sanitization:** the discovery extractor sanitizes linked-service secrets during extraction before they enter the metadata repository.
- **Prompt-injection awareness:** the Context Builder strictly bounds LLM input to compressed, structured metadata rather than raw user-controlled Azure content, reducing prompt-injection surface (documented hardening objective in `Documentation/Phases/05_Phase_5_Product_Hardening.md`).
- **Session storage:** OAuth session state is currently in-process memory (see `pie.auth.session_store` docstring) — this does **not** survive process restarts and does not horizontally scale; a Redis/Azure Cache-backed store is flagged as a future (Phase 5+) hardening item for multi-instance/production deployments.
- **CORS:** the shipped FastAPI app allows all origins (`allow_origins=["*"]`) — this is a local-development default and should be restricted before any non-local deployment.

---

12. Known Documentation Assets Already in the Repo

The project ships its own extensive internal documentation, which this document consolidates and cross-references:

File	Content
`PIE_MASTER_CONTEXT.md` / `Documentation/PIE_MASTER_CONTEXT.md`	Master architecture blueprint, all 7 phases, domain model reference, prompt library.
`Documentation/Phases/01–07_*.md`	Individual phase planning docs (Product Definition → Enterprise Roadmap).
`Documentation/Requirement/*.docx` + `all_phases_text.txt`	Original business requirement documents per phase.
`CODEBASE_ANALYSIS.md`	Prior deep-dive code analysis (1,444 lines).
`FIXES_IMPLEMENTED.md`	Log of specific bug fixes (factory-selection flow, hardcoded factory name/model persistence).
`CHAT_ERROR_DIAGNOSTICS.md`	Chat API troubleshooting guide (see the companion Runbook for an operational version).

---

13. Versioning & Status Snapshot

- Package version: **0.3.0** (`pyproject.toml`)
- API version reported at runtime: **3.0.0** (`FastAPI(version=...)` , `/health` response)
- Phase status per `PIE_MASTER_CONTEXT.md` : Phases 1–4 documented as complete/validated (including live Azure spike validation: 161 pipelines / 793 activities / 72 datasets / 58 linked services / 23 triggers / 37 data flows discovered against a real ADF instance, graph of 1,144 nodes / 2,626 edges). Phases 5–7 (hardening, demo readiness, enterprise roadmap) are documented as planning/in-progress scope — verify current completion state against `Documentation/Phases/05_Phase_5_Product_Hardening.md` onward, since planning docs can lead implementation.